Programmieren mit Python

Praktikum / Ausbildungsstation

DS49

Altenholz, Januar 2027

Inhaltsverzeichnis

1 Einrichtung der Entwicklungsumgebung 5
1.1 Der Sprachinterpreter für Python 5
1.2 VS Code: Die integrierte Entwicklungsumgebung 5
1.3 VS Code-Plugins für Python 5
1.4 Das Entwicklungsprojekt 5
1.5 Der Pythonpath 6
1.6 uv: Ein Dependency-Management-Tool für Python 6
1.7 Eine lokale virtuelle Entwicklungsumgebung . 6
1.8 pytest: Ein Testframework für Python 6
1.9 git: Eine Versionsverwaltung 7

2 Erste Schritte 8
2.1 Grundlegende Sprachelemente 8
2.2 Eine einfache Funktion 8
2.3 Die Testfunktion 8
2.4 Type Hints 9

3 Funktionen 10
3.1 Temperaturumrechnung 10
3.2 Zeichenketten 10

4 Der Zeitrechner 11

5 Die bedingte Anweisung 12
5.1 Eine Funktion zur Berechnung von Versandkosten 12
5.2 Äquivalenzklassen 12
5.3 Grenzwerte 12

5.4 Exceptions 12
5.5 Ein Wahrheitswert als Rückgabewert 12

6 Schleifen und einfache Rekursion 13
6.1 Aufgabenstellung 13
6.2 Schleifen 13
6.3 Rekursion 13

7 Sortieralgorithmen 14
7.1 Einfache Sortieralgorithmen 14
7.2 Merge Sort: Divide et impera 14

8 Aufzählungstypen 15
8.1 Typsicherheit 15
8.2 Ein Aufzählungstyp für Temperaturskalen . . 15

9 Klassen und Objekte 16
9.1 Aufgabenstellung: Bruchrechnen 16
9.2 Spoiler: Eine grafische Oberfläche 16

10 Bäume: eine rekursive Datenstruktur 17
10.1 Aufgabenstellung 17
10.2 Der binäre Baum 17
10.3 Sortieren mit Bäumen 17
10.4 Bäume ausbalancieren 17

11 Dateioperationen 18
11.1 Aufgabenstellung 18
11.2 Die Bestandsfortschreibung 18

12 Eine grafische Oberfläche 19
12.1 Aufgabenstellung 19
12.2 Das GUI-Framework NiceGui 19

13 Ein Rückblick 20

1 Einrichtung der Entwicklungsumgebung

Für das professionelle Programmieren benötigen wir die folgenden Werkzeuge.

1.1 Der Sprachinterpreter für Python

- Welche Versionen von Python sind derzeit aktuell?
- Ist auf dem Entwicklungsrechner Python installiert?
- Ist die Version gegebenenfalls aktuell?
- Installiere anderfalls die aktuellste Python-Version, die für das Betriebssystem zur Verfügung steht.

1.2 VS Code: Die integrierte Entwicklungsumgebung

- Ist auf dem Entwicklungsrechner eine aktuelle Version von VS Code installiert?
- Installiere sie andernfalls.

1.3 VS Code-Plugins für Python

- Sind in VS Code die folgenden Plugins installiert? Andernfalls bitte installieren.
  - Microsoft Python
  - Pylance
  - Python Debugger
- Falls das Plugin *Python Environments* installiert ist / wird, bitte deinstallieren. Recherchiere den Grund dafür.

1.4 Das Entwicklungsprojekt

Lege in deinem Nutzerbereich einen Ordner namens *Praktikumsprojekt* an. Lege darin folgende Dateien an:

- Eine codeworkspace-Datei für den Arbeitsbereich.
- Einen Unterordner *src* für die Quellcodedateien mit funktionalem Quellcode.
- Einen Unterordner *test* für die Quellcodedateien mit den Testfunktionen.

1.5 Der Pythonpath

VS Code benötigt Informationen über die Ordnerstruktur. Diese werden in der Datei *.env* mittels der Umgebungsvariable *PYTHONPATH* bereitsgestellt.

Lege die Datei mit den entsprechenden Einträgen an.

1.6 uv: Ein Dependency-Management-Tool für Python

Installiere das Programm uv auf dem Entwicklungsrechner.

Übergebe die Verwaltung des Projekts *Praktikumsprojekt* an *uv*.

Recherchiere:

- Welche Funktionalitäten und Vorteile besitzt *uv*?
- Welche Alternativen gibt es zu *uv*?

1.7 Eine lokale virtuelle Entwicklungsumgebung

Lege ein *Python Environment* an.

1.8 pytest: Ein Testframework für Python

Installiere mittels *uv* das Testframework *pytest* als Bibliothek für die Entwicklung.

pytest werden wir später konfigurieren.

Recherchiere: Welche Alternative gibt es zu *pytest*?

1.9 git: Eine Versionsverwaltung

Nimm das Projekt unter Verwaltung durch das lokale Git.

2 Erste Schritte

2.1 Grundlegende Sprachelemente

Informiere dich über elementare Sprachelemente in Python:

- Ausdrücke
- Befehle
- Abschnitte / Blöcke
- Datentypen
- Konstanten
- Variable
- Funktionen

2.2 Eine einfache Funktion

Was ist die Signatur einer Funktion?

Schreibe eine einfache Funktion, die

- eine Ganzzahl als Parameter entgegen nimmt und
- die Hälfte des Eingabewertes als Gleitkommazahl zurück gibt.

2.3 Die Testfunktion

Informiere dich über das Thema *Unit Test*.

- Wie funktionieren sie?
- Welche Werte müssen geprüft werden?
- Weshalb sind diese Tests gut und wichtig?

Ab sofort prüfen wir das Ergebnis eine Funktion ausschließlich mittels Unittest. Das Überprüfen mittels *print()* ist hochgradig unprofessionell.

Das systematische Testen schauen wir uns später nochmals an.

2.4 Type Hints

- Informiere dich über Type Hints.
- Stelle die Prüfung der Type Hints so ein, dass die fehlenden Typangaben für den Parameter und den Rückgabewert als Fehler angezeigt werden.
- Ergänze die Typangaben, so dass die Fehlermeldungen nicht mehr angezeigt werden.

3 Funktionen

3.1 Temperaturumrechnung

Schreibe zwei Funktionen, die jeweils Celsius in Fahrenheit bzw. umgekehrt ineinander umrechnen.

Prüfe das Ergebnis mittel Testfunktionen.

Gleitkommaarithmetik:

Bei der Umrechnung von Fahrenheit in Celsius kann ein Rundungsfehler in der Gleitkommaarithmetik auftreten.

- Identifiziere einen entsprechenden Testfall.
- Wie sollte der Test programmiert werden, dass die Rundungsdifferenz als zulässige Abweichung erkannt wird?

3.2 Zeichenketten

Schreibe eine Funktion, die für einen übergebenen String das erste Zeichen in Großschreibung umwandelt und den Rest in Kleinschreibung.

4 Der Zeitrechner

Wieviel Zeit liegt zwischen 8 Uhr 37 und 16 Uhr 13?

Schreibe eine (oder besser mehrere) Funktionen, mit deren Hilfe die Differenz zweier Uhrzeiten berechnet werden kann.

Tipp:

- Um Wertepaare zu modellieren, kennt Python den Datentyp Tupel. Informiere dich über Tupel und NamedTupel.
- Kenner dürfen auch einen eigenen Datentyp definieren. Aber das schauen wir uns später an.
- Die dritte Möglichkeit besteht in der Nutzung eines Standarddatentyps von Python.
- Probiere gerne alle drei (bzw. vier) Varianten aus.

5 Die bedingte Anweisung

5.1 Eine Funktion zur Berechnung von Versandkosten

5.2 Äquivalenzklassen

5.3 Grenzwerte

5.4 Exceptions

5.5 Ein Wahrheitswert als Rückgabewert

6 Schleifen und einfache Rekursion

6.1 Aufgabenstellung

6.2 Schleifen

6.3 Rekursion

7 Sortieralgorithmen

7.1 Einfache Sortieralgorithmen

7.2 Merge Sort: Divide et impera

8 Aufzählungstypen

8.1 Typsicherheit

8.2 Ein Aufzählungstyp für Temperaturskalen

9 Klassen und Objekte

9.1 Aufgabenstellung: Bruchrechnen

9.2 Spoiler: Eine grafische Oberfläche

10 Bäume: eine rekursive Datenstruktur

10.1 Aufgabenstellung

10.2 Der binäre Baum

10.3 Sortieren mit Bäumen

10.4 Bäume ausbalancieren

11 Dateioperationen

11.1 Aufgabenstellung

11.2 Die Bestandsfortschreibung

12 Eine grafische Oberfläche

12.1 Aufgabenstellung

12.2 Das GUI-Framework NiceGui

13 Ein Rückblick

Was hast du in diesem Praktikum bzw. dieser Station gelernt?

Erfasse 15 Punkte dazu.

Was ist für die Programmierung mit Python wichtig? Was würdest du einem Einsteiger empfehlen?

Erstelle einen kurzen Leitfaden, der ebenfalls ca. 15 Aspekte enthält.